Cambridge (CIE) A Level Computer Science

Your notes

Arrays

Contents

- Array Basics
- Array Pseudocode

Your notes

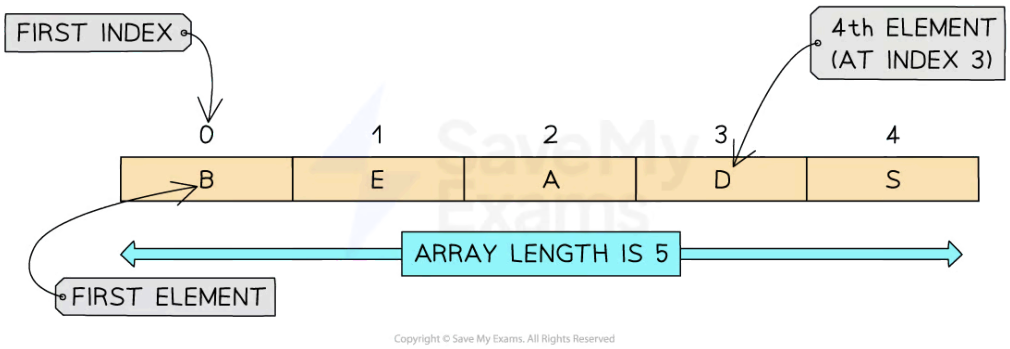
Arrays

What is an array?

- An array is an **ordered, static set of elements**
- Can only store **1 data type**
- The **position of each element** in an array is identified using the array's **index**
- The array's **first element** is the **lower bound (LB)**
- The array's **last element** is the **upper bound (UB)**
- The **lower bound of an array** is typically **0 or 1** depending on the language being used
- An array can be **one-dimensional** or **multi-dimensional**

One-dimensional (1D) arrays

- A 1D array is a **linear array**



- To declare a 1D array in pseudocode you **must include** the lower bound, upper bound and data types:

```
DECLARE <identifier> : ARRAY[LB:UB] OF <data type>
```

- In this example a 1D array of **five elements** each containing **single character** can be declared as:

```
DECLARE Letters : ARRAY[0:4] OF CHAR
```

- An example **complete program** could be:

```
// Declare the array
DECLARE Letters : ARRAY[0:4] OF CHAR

// Assign values to each index
Letters[0] ← 'B'
Letters[1] ← 'E'
Letters[2] ← 'A'
```

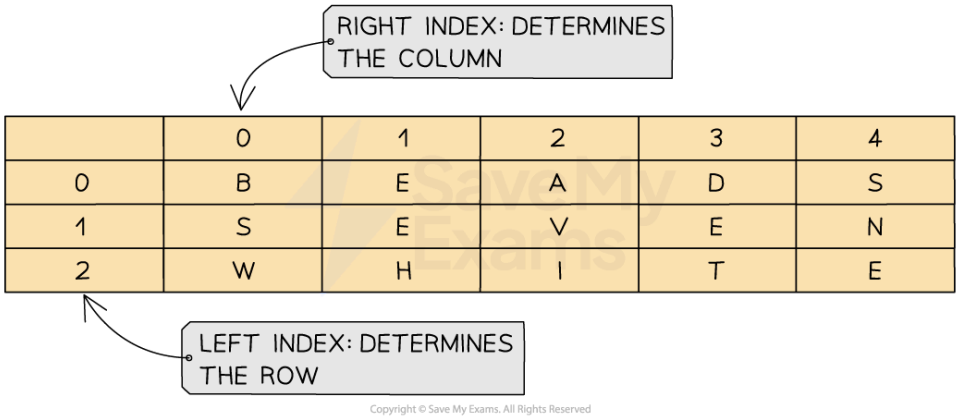
```
Letters[3] ← 'D'
Letters[4] ← 'S'

// Output the full array
FOR Index ← 0 TO 4
   OUTPUT Letters[Index]
NEXT Index
```

- The array is declared with **indices** from 0 to 4
- Each element stores a single character using the CHAR **data type**
- The **loop** outputs each letter in order

Two-dimensional (2D) arrays

- A 2D array can be **visualised as a table**
- When navigating through a 2D array you first have to go **down the rows** and then **across the columns** to find a position within the array



	0	1	2	3	4
0	B	E	A	D	S
1	S	E	V	E	N
2	W	H	I	T	E

- In 2D arrays the following must be declared:
  - Lower bound for rows (**LBR**) & upper bound for rows (**UBR**)
  - Lower bound for columns (**LBC**) & upper bound for columns (**UBC**)

```
DECLARE <identifier> : ARRAY[LBR:UBR, LBC:UBC] OF <data type>
```

- In this example a **2D array** can be declared as:

```
DECLARE Letters : ARRAY[0:2, 0:4] OF CHAR
```

- An example **complete program** could be:

```
DECLARE Letters : ARRAY[0:2, 0:4] OF CHAR

// Row 0
Grid[0,0] ← 'B'
Grid[0,1] ← 'E'
Grid[0,2] ← 'A'
Grid[0,3] ← 'D'
Grid[0,4] ← 'S'
```

Your notes

```
// Row 1
Grid[1,0] ← 'S'
Grid[1,1] ← 'E'
Grid[1,2] ← 'V'
Grid[1,3] ← 'E'
Grid[1,4] ← 'N'

// Row 2
Grid[2,0] ← 'W'
Grid[2,1] ← 'H'
Grid[2,2] ← 'I'
Grid[2,3] ← 'T'
Grid[2,4] ← 'E'
```

- ARRAY[0:2, 0:4] creates **3 rows** and **5 columns**
- First index is the **row**, second is the **column**: Letters[row, column]
- All elements are of type CHAR

Worked Example

A program reads data from a file and searches for specific data.

The main program needs to read 25 integer data items from the text file Data.txt into a local 1D array, DataArray

Write program code to declare the local array DataArray **[1]**

Answer

- 1D array with name DataArray (with 25 elements of type Integer) [1 mark]

Java

```
public static Integer[] DataArray = new Integer[25];
```

VB.NET

```
Dim DataArray(24) As Integer
```

Python

```
DataArray = [] #25 elements Integer
```

Your notes

Your notes

Searching arrays

What is a linear search?

- A linear search **starts with the first value** in a dataset and **checks every value one at a time** until all values have been checked
- A linear search can be performed **even if the values are not in order**

How do you perform a linear search?

Step	Instruction
1	Check the first value
2	**IF** it is the value you are looking for ▪ STOP!
3	**ELSE** move to the next value and check
4	**REPEAT UNTIL** you have checked all values and not found the value you are looking for

- A linear search can be used to **find elements in an array**
- Each element in the array is **checked in order**, from the **lower bound** to the **upper bound**, until the item is found or the upper bound is reached
- An example pseudocode algorithm to perform a linear search on a **1D array** could be:

```
DECLARE List : ARRAY[0:4] OF INTEGER
DECLARE Target : INTEGER
DECLARE Found : BOOLEAN
DECLARE Index : INTEGER

// Initialise array values
List[0] ← 12
List[1] ← 25
List[2] ← 37
List[3] ← 42
List[4] ← 56

// Input the value to search for
OUTPUT "Enter the value to search for:"
INPUT Target

Found ← FALSE
Index ← 0
```


© 2025 Save My Exams, Ltd.

```
WHILE Index <= 4 AND Found = FALSE DO
   IF List[Index] = Target THEN
      Found ← TRUE
   ELSE
      Index ← Index + 1
   ENDIF
ENDWHILE

IF Found = TRUE THEN
   OUTPUT "Item found at index ", Index
ELSE
   OUTPUT "Item not found"
ENDIF
```

- Works on a fixed-size array List[0:4]
- Uses a WHILE loop for flexibility (early exit if found)
- Clearly separates **input**, **processing**, and **output**

Identifier table

Identifier	Data type	Description
List	ARRAY[0:4] OF INTEGER	Stores the list of numbers to search through
Target	INTEGER	The number the user wants to search for
Found	BOOLEAN	Tracks whether the target value has been found
Index	INTEGER	The current position being checked in the array

Sorting arrays

What is a bubble sort?

- A bubble sort is a simple sorting algorithm that starts at the beginning of a dataset and **checks values** in **'pairs'** and **swaps them** if they are **not in the correct order**
- One full run of comparisons from beginning to end is called a '**pass**', a bubble sort may require **multiple** '**passes**' to sort the dataset
- The algorithm is finished when there are **no more swaps to make**

How do you perform a bubble sort?

Step	Instruction
1	Compare the first two values in the dataset

Your notes

© 2025 Save My Exams, Ltd.

2	**IF** they are in the wrong order... ▪ Swap them
3	Compare the next two values
4	**REPEAT** step 2 & 3 until you reach the end of the dataset (**pass 1**)
5	**IF** you have made any swaps... ▪ **REPEAT** from the start (pass 2,3,4...)
6	**ELSE** you have not made any swaps... ▪ STOP! the list is in the correct order

Your notes

Example

- Perform a bubble sort on the following dataset

5	2	4	1	6	3

Step	Instruction
1	Compare the first two values in the dataset 5 | 2 | 4 | 1 | 6 | 3
2	**IF** they are in the wrong order... ▪ Swap them 2 | 5 | 4 | 1 | 6 | 3
3	Compare the next two values 2 | 5 | 4 | 1 | 6 | 3
4	**REPEAT** step 2 & 3 until you reach the end of the dataset ▪ 5 & 4 SWAP! 2 | 4 | 5 | 1 | 6 | 3

	▪ 5 & 1 SWAP! 2 | 4 | 1 | 5 | 6 | 3 ▪ 5 & 6 NO SWAP! 2 | 4 | 1 | 5 | 6 | 3 ▪ 6 & 3 SWAP! 2 | 4 | 1 | 5 | 3 | 6 ▪ End of pass 1
5	**IF** you have made any swaps... ▪ **REPEAT** from the start ▪ End of pass 2 (swaps made) 2 | 1 | 4 | 3 | 5 | 6 ▪ End of pass 3 (swaps made) 1 | 2 | 3 | 4 | 5 | 6 ▪ End of pass 4 (no swaps) 1 | 2 | 3 | 4 | 5 | 6
6	**ELSE** you have not made any swaps... ▪ STOP! the list is in the correct order

- A bubble sort can be used to **put elements in an array in order**
- A **pseudocode algorithm** for a bubble sort on a 1D array could be:

```
DECLARE Numbers : ARRAY[0:5] OF INTEGER
DECLARE Temp : INTEGER
DECLARE Pass : INTEGER
DECLARE Index : INTEGER
DECLARE Swapped : BOOLEAN

// Initialise the array
Numbers[0] ← 5
Numbers[1] ← 2
Numbers[2] ← 4
```

Your notes

```
Numbers[3] ← 1
Numbers[4] ← 6
Numbers[5] ← 3

// Bubble sort algorithm
FOR Pass ← 1 TO 5
  Swapped ← FALSE
  FOR Index ← 0 TO 4
    IF Numbers[Index] > Numbers[Index + 1] THEN
      Temp ← Numbers[Index]
      Numbers[Index] ← Numbers[Index + 1]
      Numbers[Index + 1] ← Temp
      Swapped ← TRUE
    ENDIF
  NEXT Index
  IF Swapped = FALSE THEN
    // List is already sorted
    EXIT
  ENDIF
NEXT Pass

// Output the sorted array
FOR Index ← 0 TO 5
  OUTPUT Numbers[Index]
NEXT Index
```

- Uses a fixed array of size 6
- Includes an **optimisation** to exit early if no swaps were made on a pass

Identifier table

Identifier	Data type	Description
Numbers	ARRAY[0:5] OF INTEGER	Stores the list of numbers to be sorted
Temp	INTEGER	Temporary variable used for swapping two elements in the array
Pass	INTEGER	Tracks the number of passes through the array
Index	INTEGER	Tracks the current position being compared in the array
Swapped	BOOLEAN	Indicates whether a swap occurred during a pass (used to optimise sorting)

Your notes